Compte rendu — TP 1 : Découvrir la boîte à outils MLOps

Étudiante	Fatima Zahra Houqqa
Module	Déploiement d'applications intelligentes · MLOps
TP	TP 1 — Découvrir la boîte à outils MLOps

1. Travaux réalisés

L'ensemble des étapes du TP a été mené à terme : installation de l'environnement (Python, Git, environnement virtuel, MLflow, DVC), génération du jeu de données synthétique (5000 transactions, 568 fraudes), démarrage du serveur de suivi MLflow, entraînement instrumenté d'un modèle de classification (RandomForestClassifier), lancement et comparaison de cinq exécutions avec des hyperparamètres différents, enregistrement du meilleur modèle dans le registre avec l'alias « production », rechargement du modèle par cet alias, premier contact avec le versioning de données via DVC, et automatisation de l'enchaînement complet par un script PowerShell.

2. Réponses aux questions

Question 1. ***Quelle différence faites-vous entre un paramètre et une métrique dans MLflow ? Donnez un exemple de chaque.***

Un paramètre est une valeur configurée avant le lancement de l'entraînement, qui détermine le comportement de l'algorithme (par exemple le nombre d'arbres n_estimators ou la profondeur maximale max_depth d'une forêt aléatoire). Il est fixé par le développeur et reste constant pendant toute la durée de l'exécution.

Une métrique, en revanche, est une mesure calculée pendant ou après l'entraînement pour évaluer la performance du modèle (par exemple le score f1_test). Elle résulte de l'apprentissage et permet de comparer objectivement différentes exécutions.

En résumé : le paramètre est une entrée du processus d'entraînement, tandis que la métrique en est une sortie mesurable.

Question 2. ***Deux exécutions ont un F1 très proche mais des durées très différentes. Laquelle promouvez-vous, et pourquoi ?***

Lorsque deux exécutions présentent un F1-score très proche, je privilégie celle dont la durée d'entraînement est la plus courte. Un gain marginal de performance ne justifie pas un coût opérationnel plus élevé : un modèle plus rapide à entraîner est aussi plus économique à réentraîner et généralement plus réactif en production.

Question 3. ***À quoi sert l'alias production plutôt que le numéro de version dans le code du service d'inférence ?***

L'alias production permet au service d'inférence de charger toujours le modèle actuellement en service, sans connaître ni modifier son numéro de version. Lorsqu'un nouveau modèle est validé, il suffit de déplacer l'alias vers la nouvelle version dans le registre ; le code du service reste inchangé, ce qui évite tout redéploiement et réduit les risques d'erreur liés à une mise à jour manuelle du numéro de version.

Question 4. ***Que contient exactement le fichier .dvc, et pourquoi Git ne doit-il pas suivre le CSV lui-même ?***

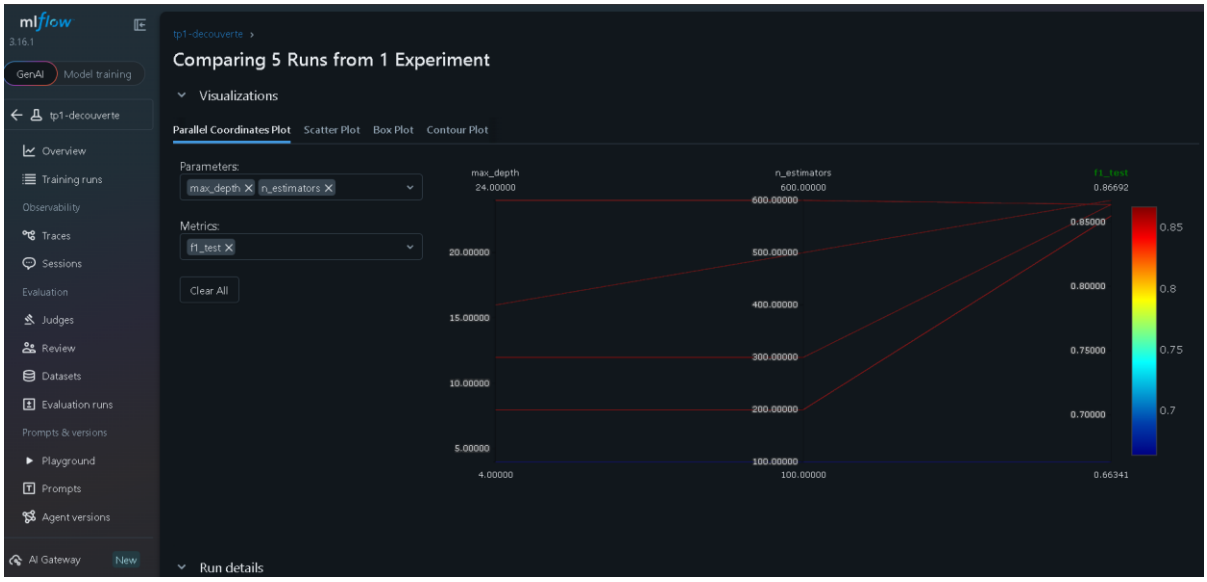
Le fichier .dvc ne contient pas les données elles-mêmes, mais un pointeur léger : l'empreinte MD5 du fichier, sa taille en octets et son chemin d'origine. Git ne doit pas suivre directement le fichier CSV car les données volumineuses alourdiraient excessivement le dépôt et rendraient son historique difficile à gérer. En déléguant le suivi du fichier réel à DVC, Git ne conserve que ce pointeur de quelques dizaines d'octets, tandis que DVC gère le stockage et le versionnement des données elles-mêmes.

Question 5. ***Citez deux informations qu'il faudrait journaliser en plus pour qu'une exécution soit vraiment reproductible.***

Pour garantir une reproductibilité complète, il faudrait également journaliser l'empreinte (hash) du jeu de données utilisé, afin de s'assurer que les mêmes données exactes ont servi à l'entraînement, ainsi que les versions précises des bibliothèques logicielles employées (scikit-learn, pandas, Python), car une différence de version peut modifier légèrement le comportement des algorithmes et donc le résultat obtenu.

Annexe — Captures d'écran

Comparaison des cinq exécutions (métrique f1_test)



Registre des modèles — version enregistrée et alias « production »

